📋 Daftar Isi

1. Setup & Import Libraries
2. Data Loading & Eksplorasi
3. Data Preprocessing
   - 3.1 Deteksi Outlier (IQR)
   - 3.2 Standarisasi Fitur (StandardScaler)
4. Penentuan Jumlah Cluster Optimal
   - 4.1 Elbow Method
   - 4.2 Silhouette Analysis
5. K-Means Clustering
6. Ward's Hierarchical Clustering
   - 6.1 Dendrogram
   - 6.2 Penerapan & Validasi
7. Visualisasi PCA 2D
8. Profil Cluster & Pelabelan Risiko
9. Rekomendasi Strategis

1. Setup & Import Libraries

```
# ── Core ──────────────────────────────────────────────────────────
import glob
import os
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np

# ── Visualisasi ───────────────────────────────────────────────────
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.cm as cm
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# ── Machine Learning ──────────────────────────────────────────────
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, silhouette_samples
from sklearn.metrics import adjusted_rand_score

# ── Hierarchical Clustering ───────────────────────────────────────
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from scipy.spatial.distance import cdist

# ── Global Aesthetics ─────────────────────────────────────────────
plt.rcParams.update({
    'figure.dpi'      : 130,
    'font.family'     : 'DejaVu Sans',
    'axes.titlesize'  : 13,
    'axes.labelsize'  : 11,
    'axes.spines.top' : False,
    'axes.spines.right': False,
})
PALETTE     = sns.color_palette('husl', 6)
RANDOM_STATE = 42

print("✅ Semua library berhasil dimuat.")
print(f"   Pandas     : {pd.__version__}")
print(f"   NumPy      : {np.__version__}")
print(f"   Scikit-learn sudah siap.")
```

```
✅ Semua library berhasil dimuat.
   Pandas     : 2.3.3
```

```
NumPy      : 2.4.1
Scikit-learn sudah siap.
```

2. Data Loading & Eksplorasi

Data diambil dari **5 file Excel** (2020–2024) yang masing-masing berisi 38 Kabupaten/Kota Jawa Timur. Baris dengan kata kunci **"Jawa Timur"** (ringkasan provinsi) **diabaikan** sesuai instruksi. Nilai agregasi multi-tahun menggunakan **median** untuk ketahanan terhadap anomali data.

```
# ── Konfigurasi ──────────────────────────────────────────────────────────
DATA_DIR  = "tingkat kriminalitas"
FILES     = sorted(glob.glob(os.path.join(DATA_DIR, "*.xlsx")))

COL_NAMES    = ['Kabupaten_Kota', 'Jumlah_Kejahatan', 'Risiko_100k',
                'Persen_Penyelesaian', 'Selang_Waktu']
FEATURE_COLS = COL_NAMES[1:]

FEATURE_LABELS = {
    'Jumlah_Kejahatan'   : 'Jml Kejahatan\nDilaporkan',
    'Risiko_100k'        : 'Risiko per\n100k Penduduk',
    'Persen_Penyelesaian': '% Penyelesaian\nKasus',
    'Selang_Waktu'       : 'Selang Waktu\nKejadian (jam)',
}

print(f"📂 Ditemukan {len(FILES)} file:")
for f in FILES:
    print(f"   • {os.path.basename(f)}")
```

```
📂 Ditemukan 5 file:
   • tingkat kriminalitas 2020.xlsx
   • tingkat kriminalitas 2021.xlsx
   • tingkat kriminalitas 2022.xlsx
   • tingkat kriminalitas 2023.xlsx
   • tingkat kriminalitas 2024.xlsx
```

```
def load_year_data(filepath: str) -> pd.DataFrame:
    """
    Memuat satu file Excel, menghapus baris 'Jawa Timur', baris metadata/catatan,
    dan mengkonversi semua kolom numerik.
    """
    year = int(os.path.basename(filepath).split()[-1].replace('.xlsx', ''))
    df   = pd.read_excel(filepath, header=0)
    df.columns = COL_NAMES

    # ① Filter baris 'Jawa Timur' (case-insensitive)
    mask_jatim = df['Kabupaten_Kota'].astype(str).str.lower().str.contains('jawa timur', na=True)
    df = df[~mask_jatim].copy()

    # ② Konversi ke numerik; nilai tidak valid → NaN
    for col in FEATURE_COLS:
        df[col] = pd.to_numeric(df[col], errors='coerce')

    # ③ Hapus baris metadata (catatan, sumber, HTML-tag, dll.)
    df = df[df['Kabupaten_Kota'].notna()].copy()
    df = df[~df['Kabupaten_Kota'].astype(str).str.match(
        r'^[<\d]|catatan|sumber|source|note', flags=0, na=True
    )].copy()

    # ④ Hanya simpan baris dengan ≥ 3 kolom numerik valid
    valid_mask = df[FEATURE_COLS].notna().sum(axis=1) >= 3
    df = df[valid_mask].copy()

    df['Tahun'] = year
    return df


# ── Load & gabungkan ─────────────────────────────────────────────────────
dfs    = [load_year_data(f) for f in FILES]
df_raw = pd.concat(dfs, ignore_index=True)
df_raw[FEATURE_COLS] = df_raw[FEATURE_COLS].apply(pd.to_numeric, errors='coerce')

print(f"✅ Total rekaman: {len(df_raw)}")
print(f"   Tahun     : {sorted(df_raw['Tahun'].unique())}")
print(f"   Kabupaten : {df_raw['Kabupaten_Kota'].nunique()} daerah unik")
df_raw.head(10)
```

```
✅ Total rekaman: 190
   Tahun     : [np.int64(2020), np.int64(2021), np.int64(2022), np.int64(2023), np.int64(2024)]
   Kabupaten : 38 daerah unik
```

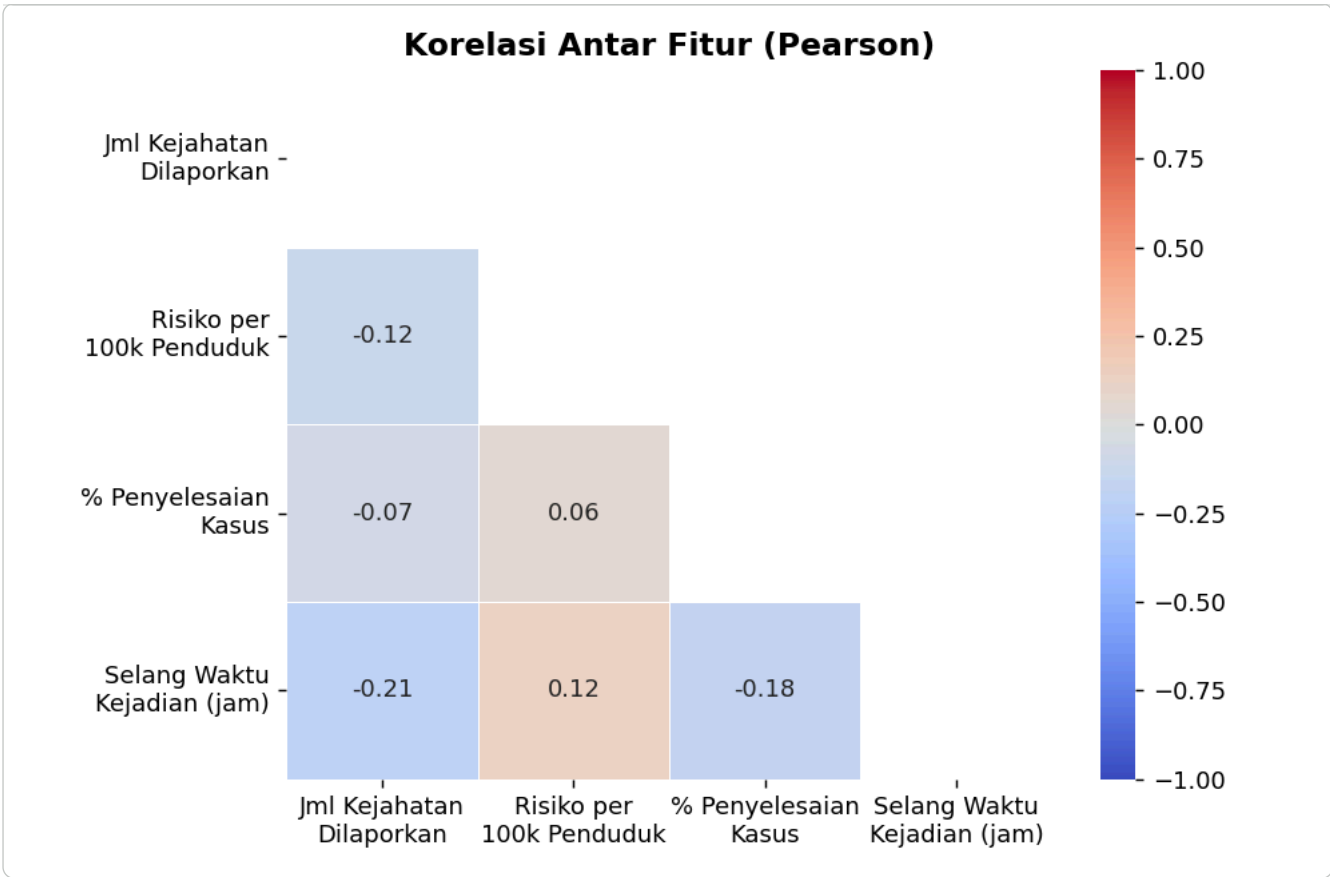
	Kabupaten_Kota	Jumlah_Kejahatan	Risiko_100k	Persen_Penyelesaian	Selang_Waktu	Tahun
0	Pacitan	72.0	13.0	90.28	100.0	2020
1	Ponorogo	402.0	47.0	87.81	1307.0	2020
2	Trenggalek	595.0	88.0	90.59	883.0	2020
3	Tulungagung	538.0	54.0	100.74	977.0	2020
4	Blitar	304.0	27.0	27.30	289.0	2020
5	Kediri	862.0	57.0	38.75	610.0	2020
6	Malang	1850.0	75.0	69.35	284.0	2020
7	Lumajang	356.0	35.0	12.64	36.0	2020
8	Jember	1061.0	45.0	62.49	495.0	2020
9	Banyuwangi	893.0	57.0	47.70	589.0	2020

```
# ── Agregasi multi-tahun: median per Kabupaten/Kota ──────────────────────────
# Median lebih robust terhadap anomali data tahun tertentu (mis. 2024)
df_profile = (df_raw
              .groupby('Kabupaten_Kota', sort=True)[FEATURE_COLS]
              .median()
              .reset_index())

print(f"✅ Dataset profil final: {df_profile.shape[0]} Kabupaten/Kota × {len(FEATURE_COLS)} fitur")
print("\n📊 Statistik Deskriptif:")
df_profile[FEATURE_COLS].describe().round(2)
```

```
✅ Dataset profil final: 38 Kabupaten/Kota × 4 fitur

📊 Statistik Deskriptif:
```

	Jumlah_Kejahatan	Risiko_100k	Persen_Penyelesaian	Selang_Waktu
count	38.00	38.00	38.00	38.00
mean	1122.82	54.32	86.18	657.27
std	441.28	47.67	13.02	239.75
min	300.00	6.00	46.42	231.60
25%	764.25	28.50	81.60	447.00
50%	1057.50	42.50	86.24	682.00
75%	1501.75	56.25	95.29	835.75
max	2009.00	222.00	109.16	1215.00

```
# ── Pearson Correlation Heatmap ──────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(7, 5))
corr = df_profile[FEATURE_COLS].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))

sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            vmin=-1, vmax=1, ax=ax, linewidths=0.5,
            xticklabels=list(FEATURE_LABELS.values()),
            yticklabels=list(FEATURE_LABELS.values()))
ax.set_title('Korelasi Antar Fitur (Pearson)', fontweight='bold')
plt.tight_layout()
plt.savefig('correlation_matrix.png', bbox_inches='tight', dpi=130)
plt.show()
```



3. Data Preprocessing

3.1 Deteksi & Penanganan Outlier — Metode IQR

Nilai ekstrem diidentifikasi dengan batas **IQR × 1.5** dan dikap (*Winsorization*) agar jumlah Kabupaten/Kota tetap 38 data point.

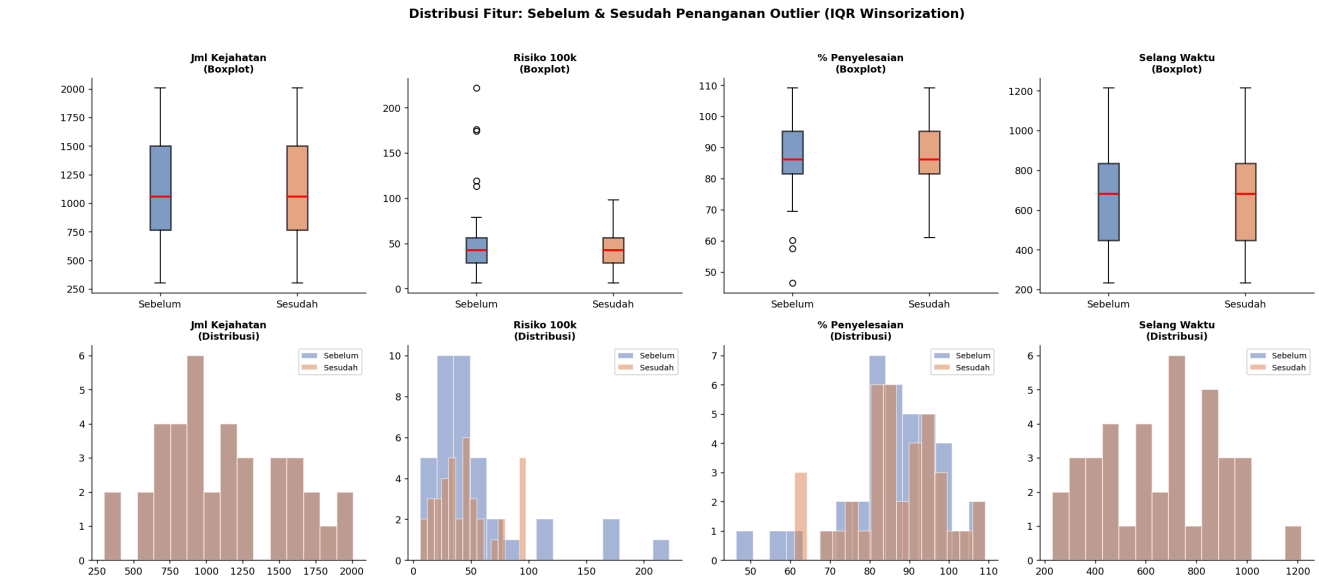
Arah Ekstrem	Interpretasi Risiko
`Jumlah_Kejahatan` tinggi	Beban kasus besar
`Risiko_100k` tinggi	Kerawanan wilayah tinggi
`Persen_Penyelesaian` sangat rendah	Kapasitas aparat menurun
`Selang_Waktu` sangat pendek	Frekuensi kejadian sangat tinggi

```
def detect_and_cap_iqr(df: pd.DataFrame, cols: list) -> tuple[pd.DataFrame, dict]:
    """
    Deteksi outlier IQR dan terapkan Winsorization (capping).
    Returns: (df_clean, report_dict)
    """
    df_out = df.copy()
    report = {}

    for col in cols:
        q1   = df[col].quantile(0.25)
        q3   = df[col].quantile(0.75)
        iqr  = q3 - q1
        lo   = q1 - 1.5 * iqr
        hi   = q3 + 1.5 * iqr

        outlier_idx  = df[(df[col] < lo) | (df[col] > hi)].index
        outlier_kota = df.loc[outlier_idx, 'Kabupaten_Kota'].tolist()

        df_out[col] = df_out[col].clip(lower=lo, upper=hi)
        report[col] = dict(Q1=q1, Q3=q3, IQR=iqr,
                           lower=lo, upper=hi,
                           outliers=outlier_kota,
                           n_outliers=len(outlier_kota))
    return df_out, report


df_clean, iqr_report = detect_and_cap_iqr(df_profile, FEATURE_COLS)

# ── Cetak laporan ──────────────────────────────────────────────────────
print("=" * 65)
print("  LAPORAN DETEKSI OUTLIER — METODE IQR (Threshold = 1.5×IQR)")
print("=" * 65)
```

```
for col, info in iqr_report.items():
    status = f"⚠️  {info['n_outliers']} daerah" if info['n_outliers'] else "✅ Tidak ada"
    print(f"\n  {col}")
    print(f"    Batas bawah : {info['lower']:>10.2f}  |  Batas atas : {info['upper']:>10.2f}")
    print(f"    Outlier     : {status}")
    if info['outliers']:
        print(f"    Daerah      : {', '.join(info['outliers'])}")
```

```
==================================================================
  LAPORAN DETEKSI OUTLIER — METODE IQR (Threshold = 1.5×IQR)
==================================================================

  Jumlah_Kejahatan
    Batas bawah :    -342.00  |  Batas atas :    2608.00
    Outlier     : ✅ Tidak ada

  Risiko_100k
    Batas bawah :     -13.12  |  Batas atas :      97.88
    Outlier     : ⚠️  5 daerah
    Daerah      : Kota Kediri, Kota Mojokerto, Kota Pasuruan, Kota Probolinggo, Magetan

  Persen_Penyelesaian
    Batas bawah :      61.07  |  Batas atas :     115.83
    Outlier     : ⚠️  3 daerah
    Daerah      : Kota Malang, Situbondo, Sumenep

  Selang_Waktu
    Batas bawah :    -136.12  |  Batas atas :    1418.88
    Outlier     : ✅ Tidak ada
```

```
# ── Visualisasi Before vs After Winsorization ──────────────────────────────
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
short_labels = ['Jml Kejahatan', 'Risiko 100k', '% Penyelesaian', 'Selang Waktu']
palette_box  = ['#4C72B0', '#DD8452']

for i, (col, lbl) in enumerate(zip(FEATURE_COLS, short_labels)):
    # Boxplot Before / After
    ax_box = axes[0, i]
    bp = ax_box.boxplot(
        [df_profile[col].dropna().values, df_clean[col].dropna().values],
        labels=['Sebelum', 'Sesudah'], patch_artist=True,
        medianprops=dict(color='red', linewidth=2),
        boxprops=dict(linewidth=1.5),
    )
    for patch, color in zip(bp['boxes'], palette_box):
        patch.set_facecolor(color)
        patch.set_alpha(0.7)
    ax_box.set_title(f'{lbl}\n(Boxplot)', fontsize=10, fontweight='bold')

    # Histogram
    ax_hist = axes[1, i]
    ax_hist.hist(df_profile[col].dropna(), bins=15, alpha=0.5,
                 color=palette_box[0], label='Sebelum', edgecolor='white')
    ax_hist.hist(df_clean[col].dropna(), bins=15, alpha=0.5,
                 color=palette_box[1], label='Sesudah', edgecolor='white')
    ax_hist.set_title(f'{lbl}\n(Distribusi)', fontsize=10, fontweight='bold')
    ax_hist.legend(fontsize=8)

fig.suptitle('Distribusi Fitur: Sebelum & Sesudah Penanganan Outlier (IQR Winsorization)',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('outlier_handling.png', bbox_inches='tight', dpi=130)
plt.show()
```

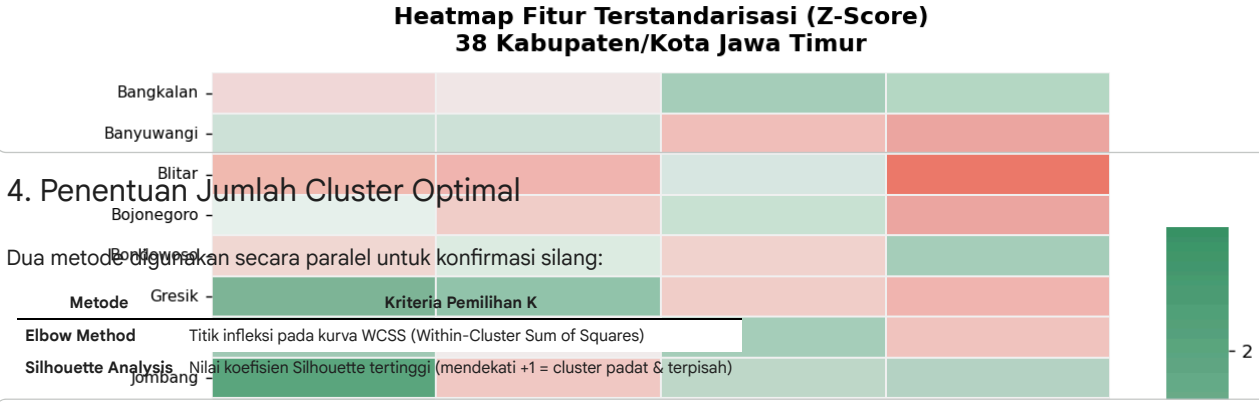


3.2 Standarisasi Fitur — StandardScaler (Z-Score Normalization)

StandardScaler mengubah setiap fitur menjadi distribusi dengan **mean = 0** dan **std = 1**, sehingga perbedaan skala antar variabel tidak mendominasi perhitungan jarak cluster.

```
scaler  = StandardScaler()
X_scaled = scaler.fit_transform(df_clean[FEATURE_COLS])
X_df     = pd.DataFrame(X_scaled, columns=FEATURE_COLS,
                        index=df_clean['Kabupaten_Kota'].values)

print("Verifikasi StandardScaler:")
stat_df = pd.DataFrame({
    'Mean (harus ≈ 0)': X_scaled.mean(axis=0).round(6),
    'Std  (harus ≈ 1)': X_scaled.std(axis=0).round(6),
}, index=FEATURE_COLS)
print(stat_df.to_string())

# ── Heatmap Fitur Terstandarisasi ──────────────────────────────────────
fig, ax = plt.subplots(figsize=(10, 13))
cmap    = sns.diverging_palette(15, 145, s=80, l=55, as_cmap=True)

sns.heatmap(X_df,
            cmap=cmap, center=0, vmin=-2.5, vmax=2.5,
            annot=False, linewidths=0.25, linecolor='#f0f0f0',
            xticklabels=list(FEATURE_LABELS.values()),
            yticklabels=X_df.index, ax=ax,
            cbar_kws={'label': 'Z-Score', 'shrink': 0.8})

ax.set_title('Heatmap Fitur Terstandarisasi (Z-Score)\n38 Kabupaten/Kota Jawa Timur',
             fontweight='bold', pad=12)
ax.set_xlabel('')
ax.tick_params(axis='y', labelsize=8.5)
ax.tick_params(axis='x', labelsize=9, rotation=0)
plt.tight_layout()
plt.savefig('feature_heatmap.png', bbox_inches='tight', dpi=130)
plt.show()
```

```
Verifikasi StandardScaler:
                      Mean (harus ≈ 0)  Std  (harus ≈ 1)
Jumlah_Kejahatan                   0.0               1.0
Risiko_100k                       -0.0               1.0
Persen_Penyelesaian               -0.0               1.0
Selang_Waktu                       0.0               1.0
```

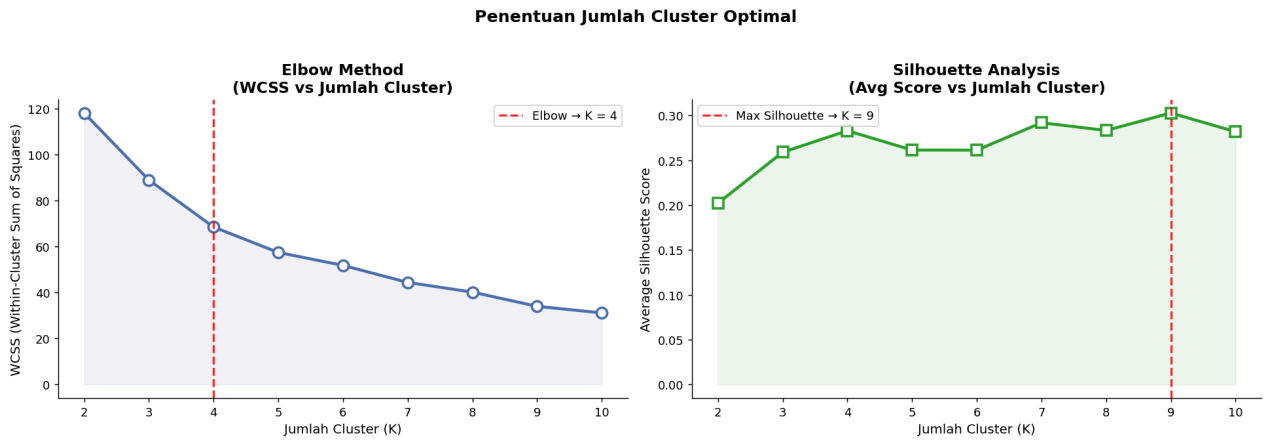


4. Penentuan Jumlah Cluster Optimal

Dua metode digunakan secara paralel untuk konfirmasi silang:

Metode	Kriteria Pemilihan K
Elbow Method	Titik infleksi pada kurva WCSS (Within-Cluster Sum of Squares)
Silhouette Analysis	Nilai koefisien Silhouette tertinggi (mendekati +1 = cluster padat & terpisah)

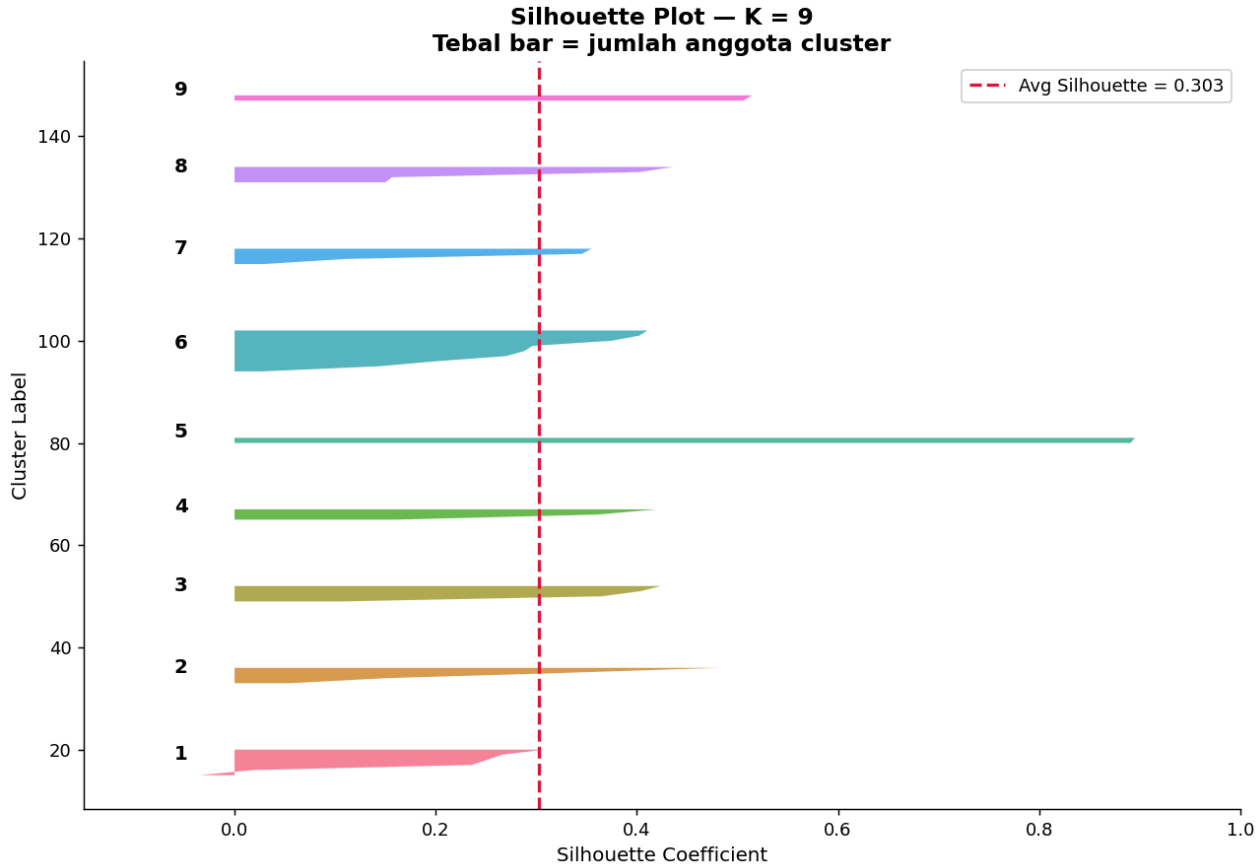
```
K_RANGE = range(2, 11)
wcss_list   = []
silh_list   = []

for k in K_RANGE:
    km = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=25, max_iter=500)
    km.fit(X_scaled)
    wcss_list.append(km.inertia_)
    silh_list.append(silhouette_score(X_scaled, km.labels_))

# ── Deteksi Elbow otomatis (maximum curvature / second derivative) ──────────────
diffs1 = np.diff(wcss_list)
diffs2 = np.diff(diffs1)
elbow_k = list(K_RANGE)[np.argmax(diffs2) + 1]

# ── K dengan Silhouette tertinggi ──────────────────────────────────────────────
best_silh_k = list(K_RANGE)[np.argmax(silh_list)]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# — Elbow —
ax1.plot(K_RANGE, wcss_list, 'o-', color='#4C72B0', linewidth=2.5, markersize=9,
         markerfacecolor='white', markeredgewidth=2)
ax1.fill_between(K_RANGE, wcss_list, alpha=0.08, color='#4C72B0')
ax1.axvline(x=elbow_k, color='red', linestyle='--', linewidth=1.8, alpha=0.85,
            label=f'Elbow → K = {elbow_k}')
ax1.set_title('Elbow Method\n(WCSS vs Jumlah Cluster)', fontweight='bold')
ax1.set_xlabel('Jumlah Cluster (K)')
ax1.set_ylabel('WCSS (Within-Cluster Sum of Squares)')
ax1.legend()
ax1.set_xticks(list(K_RANGE))

# — Silhouette —
ax2.plot(K_RANGE, silh_list, 's-', color='#2CA02C', linewidth=2.5, markersize=9,
         markerfacecolor='white', markeredgewidth=2)
ax2.fill_between(K_RANGE, silh_list, alpha=0.08, color='#2CA02C')
ax2.axvline(x=best_silh_k, color='red', linestyle='--', linewidth=1.8, alpha=0.85,
            label=f'Max Silhouette → K = {best_silh_k}')
ax2.set_title('Silhouette Analysis\n(Avg Score vs Jumlah Cluster)', fontweight='bold')
ax2.set_xlabel('Jumlah Cluster (K)')
ax2.set_ylabel('Average Silhouette Score')
ax2.legend()
ax2.set_xticks(list(K_RANGE))

plt.suptitle('Penentuan Jumlah Cluster Optimal', fontsize=14, fontweight='bold', y=1.03)
plt.tight_layout()
plt.savefig('optimal_k.png', bbox_inches='tight', dpi=130)
plt.show()

print(f"  Elbow Method       → K = {elbow_k}")
print(f"  Silhouette Analysis → K = {best_silh_k}")
```




```
Elbow Method       → K = 4
Silhouette Analysis → K = 9
```

```
# ── Gunakan mayoritas / Silhouette sebagai acuan utama ──────────────────────
OPTIMAL_K = best_silh_k   # ← Ubah manual bila perlu
print(f"\n🎯 Jumlah Cluster yang Digunakan: K = {OPTIMAL_K}")

# ── Silhouette Plot Detail untuk OPTIMAL_K ──────────────────────────────────
km_tmp    = KMeans(n_clusters=OPTIMAL_K, random_state=RANDOM_STATE, n_init=25)
lbl_tmp   = km_tmp.fit_predict(X_scaled)
silh_vals = silhouette_samples(X_scaled, lbl_tmp)
avg_silh  = silhouette_score(X_scaled, lbl_tmp)

fig, ax = plt.subplots(figsize=(10, 7))
y_lower  = 15
col_list = sns.color_palette('husl', OPTIMAL_K)

for i in range(OPTIMAL_K):
    ith  = np.sort(silh_vals[lbl_tmp == i])
    y_upper = y_lower + len(ith)
    ax.fill_betweenx(np.arange(y_lower, y_upper), 0, ith,
                     facecolor=col_list[i], edgecolor='none', alpha=0.85)
    ax.text(-0.06, y_lower + 0.5 * len(ith), str(i + 1), fontsize=11, fontweight='bold')
    y_lower = y_upper + 12

ax.axvline(x=avg_silh, color='crimson', linestyle='--', linewidth=1.8,
           label=f'Avg Silhouette = {avg_silh:.3f}')
ax.set_xlim([-0.15, 1.0])
ax.set_title(f'Silhouette Plot — K = {OPTIMAL_K}\nTebal bar = jumlah anggota cluster',
             fontweight='bold')
ax.set_xlabel('Silhouette Coefficient')
ax.set_ylabel('Cluster Label')
ax.legend(fontsize=10)
plt.tight_layout()
plt.savefig('silhouette_detail.png', bbox_inches='tight', dpi=130)
plt.show()
```

```
🎯 Jumlah Cluster yang Digunakan: K = 9
```



5. K-Means Clustering

K-Means meminimalkan **WCSS** dengan memperbarui centroid secara iteratif. Dijalankan dengan `n_init=25` untuk menghindari konvergensi ke local optimum.

```
kmeans = KMeans(n_clusters=OPTIMAL_K, random_state=RANDOM_STATE,
                n_init=25, max_iter=500)
df_clean = df_clean.copy()
df_clean['Cluster_KMeans'] = kmeans.fit_predict(X_scaled)

silh_km = silhouette_score(X_scaled, df_clean['Cluster_KMeans'])
print(f"K-Means Silhouette Score  : {silh_km:.4f}")
print(f"K-Means WCSS (Inertia)    : {kmeans.inertia_:.2f}")
print(f"Iterasi (Konvergensi)     : {kmeans.n_iter_}")
print()
print("┌──────────────────────────────────────────────────────────────────────┐")
print("  ANGGOTA CLUSTER — K-MEANS")
print("└──────────────────────────────────────────────────────────────────────┘")
for cl_id in sorted(df_clean['Cluster_KMeans'].unique()):
    members = df_clean[df_clean['Cluster_KMeans'] == cl_id]['Kabupaten_Kota'].tolist()
    print(f"  Cluster {cl_id + 1:>2} ({len(members):>2} daerah): {', '.join(members)}")
```

```
K-Means Silhouette Score  : 0.3027
K-Means WCSS (Inertia)    : 34.03
Iterasi (Konvergensi)     : 5

┌──────────────────────────────────────────────────────────────────────┐
  ANGGOTA CLUSTER — K-MEANS
└──────────────────────────────────────────────────────────────────────┘
  Cluster  1 ( 6 daerah): Bondowoso, Kota Malang, Mojokerto, Pamekasan, Sumenep, Tuban
```

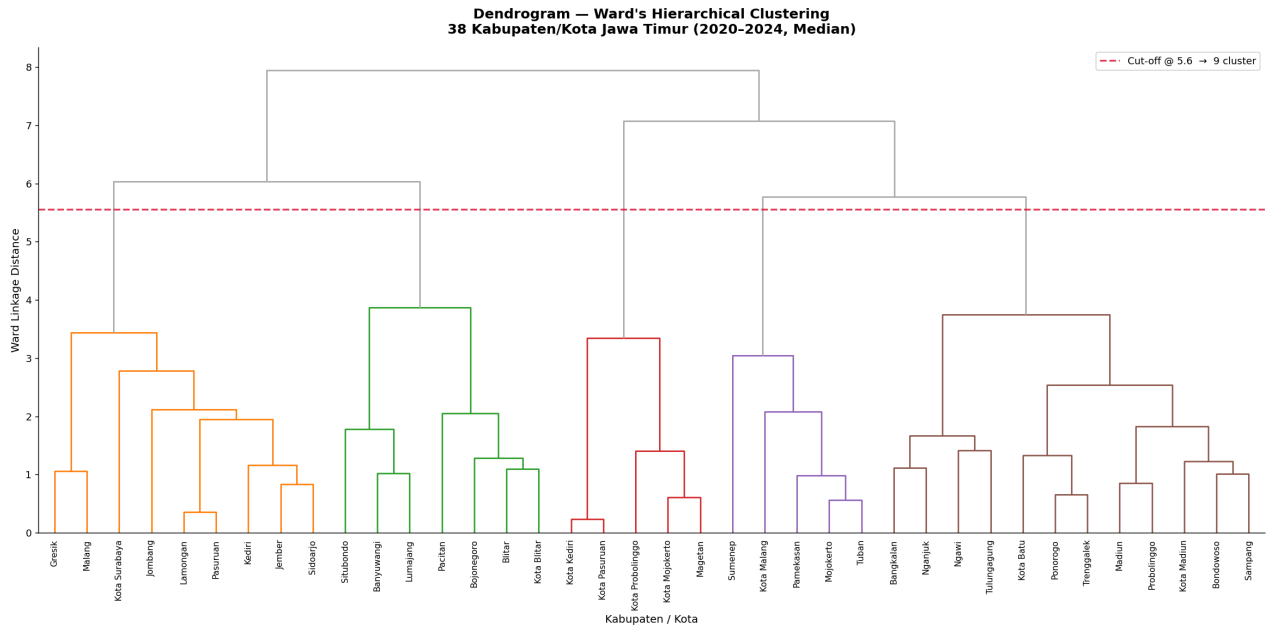
```
  Cluster  2 ( 4 daerah): Blitar, Bojonegoro, Kota Blitar, Pacitan
  Cluster  3 ( 4 daerah): Kota Mojokerto, Kota Probolinggo, Magetan, Ngawi
  Cluster  4 ( 3 daerah): Banyuwangi, Lumajang, Situbondo
  Cluster  5 ( 2 daerah): Kota Kediri, Kota Pasuruan
  Cluster  6 ( 9 daerah): Bangkalan, Kota Batu, Kota Madiun, Madiun, Nganjuk, Ponorogo, Probolinggo, Sampang, Tre
  Cluster  7 ( 4 daerah): Jember, Kota Surabaya, Sidoarjo, Tulungagung
  Cluster  8 ( 4 daerah): Jombang, Kediri, Lamongan, Pasuruan
  Cluster  9 ( 2 daerah): Gresik, Malang
```

6. Ward's Hierarchical Clustering

6.1 Dendrogram

Dendrogram menampilkan hierarki penggabungan cluster dari bawah ke atas. **Ward's Linkage** meminimalkan varians total dalam cluster saat penggabungan.

```
# ── Hitung Ward Linkage Matrix ──────────────────────────────────────────
Z_ward = linkage(X_scaled, method='ward', metric='euclidean')

# ── Ambang potong otomatis (70% dari jarak maksimum) ────────────────────
threshold = 0.70 * max(Z_ward[:, 2])

fig, ax = plt.subplots(figsize=(18, 9))
dendrogram(
    Z_ward,
    labels      = df_clean['Kabupaten_Kota'].values,
    orientation = 'top',
    leaf_rotation  = 90,
    leaf_font_size = 8.5,
    color_threshold = threshold,
    above_threshold_color = '#AAAAAA',
    ax = ax,
)
ax.axhline(y=threshold, color='crimson', linestyle='--', linewidth=1.8, alpha=0.85,
           label=f'Cut-off @ {threshold:.1f}  →  {OPTIMAL_K} cluster')
ax.set_title(
    "Dendrogram — Ward's Hierarchical Clustering\n38 Kabupaten/Kota Jawa Timur (2020–2024, Median)",
    fontsize=14, fontweight='bold', pad=14
)
ax.set_xlabel('Kabupaten / Kota', fontsize=11)
ax.set_ylabel('Ward Linkage Distance', fontsize=11)
ax.legend(fontsize=10)
plt.tight_layout()
plt.savefig('dendrogram.png', bbox_inches='tight', dpi=130)
plt.show()
```



6.2 Penerapan Hierarchical Clustering & Validasi Silang

```
# ── Terapkan AgglomerativeClustering ──────────────────────────────────
hc = AgglomerativeClustering(n_clusters=OPTIMAL_K, linkage='ward')
df_clean['Cluster_HC'] = hc.fit_predict(X_scaled)

silh_hc = silhouette_score(X_scaled, df_clean['Cluster_HC'])
print(f"Hierarchical (Ward) Silhouette Score : {silh_hc:.4f}")

# ── Adjusted Rand Index — kesepakatan dua metode ──────────────────────
ari = adjusted_rand_score(df_clean['Cluster_KMeans'], df_clean['Cluster_HC'])
print(f"Adjusted Rand Index (K-Means vs HC)   : {ari:.4f}")
print("  (ARI = 1.0 → identik sempurna | 0.0 → tidak ada kesepakatan)")

print()
print("┌──────────────────────────────────────────────────────────────────┐")
print("  ANGGOTA CLUSTER — WARD'S HIERARCHICAL")
print("└──────────────────────────────────────────────────────────────────┘")
for cl_id in sorted(df_clean['Cluster_HC'].unique()):
    members = df_clean[df_clean['Cluster_HC'] == cl_id]['Kabupaten_Kota'].tolist()
    print(f"  Cluster {cl_id + 1:>2} ({len(members):>2} daerah): {', '.join(members)}")
```

```
Hierarchical (Ward) Silhouette Score : 0.2935
Adjusted Rand Index (K-Means vs HC)   : 0.6525
  (ARI = 1.0 → identik sempurna | 0.0 → tidak ada kesepakatan)

┌──────────────────────────────────────────────────────────────────┐
  ANGGOTA CLUSTER — WARD'S HIERARCHICAL
└──────────────────────────────────────────────────────────────────┘
  Cluster  1 ( 5 daerah): Kota Malang, Mojokerto, Pamekasan, Sumenep, Tuban
  Cluster  2 ( 8 daerah): Bondowoso, Kota Batu, Kota Madiun, Madiun, Ponorogo, Probolinggo, Sampang, Trenggalek
  Cluster  3 ( 7 daerah): Jember, Jombang, Kediri, Kota Surabaya, Lamongan, Pasuruan, Sidoarjo
  Cluster  4 ( 4 daerah): Blitar, Bojonegoro, Kota Blitar, Pacitan
  Cluster  5 ( 3 daerah): Kota Mojokerto, Kota Probolinggo, Magetan
  Cluster  6 ( 3 daerah): Banyuwangi, Lumajang, Situbondo
  Cluster  7 ( 4 daerah): Bangkalan, Nganjuk, Ngawi, Tulungagung
  Cluster  8 ( 2 daerah): Gresik, Malang
  Cluster  9 ( 2 daerah): Kota Kediri, Kota Pasuruan
```

```
# ── Tabel perbandingan keanggotaan ────────────────────────────────────
comparison = df_clean[['Kabupaten_Kota', 'Cluster_KMeans', 'Cluster_HC']].copy()
comparison['Cluster_KMeans_Label'] = comparison['Cluster_KMeans'] + 1
comparison['Cluster_HC_Label']     = comparison['Cluster_HC'] + 1
comparison['Kesepakatan']          = (comparison['Cluster_KMeans'] == comparison['Cluster_HC']).map(
    {True: '✅', False: '⚠️ Beda'}
)
```

```
# Confusion Matrix K-Means vs HC
from pandas import crosstab
cm_df = crosstab(comparison['Cluster_KMeans_Label'],
                 comparison['Cluster_HC_Label'],
                 rownames=['K-Means'], colnames=['Hierarchical'])
print("\nConfusion Matrix Keanggotaan Cluster:")
print(cm_df.to_string())
```

```
Confusion Matrix Keanggotaan Cluster:
Hierarchical  1  2  3  4  5  6  7  8  9
K-Means
1             5  1  0  0  0  0  0  0  0
2             0  0  0  4  0  0  0  0  0
3             0  0  0  0  3  0  1  0  0
4             0  0  0  0  0  3  0  0  0
5             0  0  0  0  0  0  0  0  2
6             0  7  0  0  0  0  2  0  0
7             0  0  3  0  0  0  1  0  0
8             0  0  4  0  0  0  0  0  0
9             0  0  0  0  0  0  0  2  0
```

7. Visualisasi PCA 2D

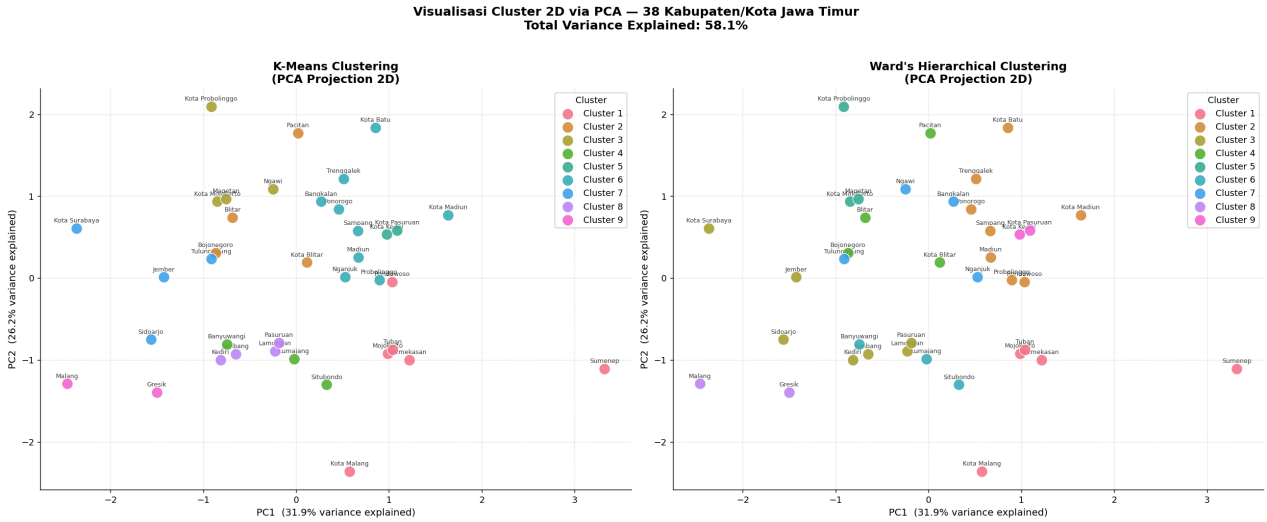
PCA (Principal Component Analysis) mereduksi dimensi 4D → 2D untuk visualisasi. Setiap titik mewakili satu Kabupaten/Kota; warna menunjukkan cluster.

```
pca  = PCA(n_components=2, random_state=RANDOM_STATE)
X_pc = pca.fit_transform(X_scaled)

ev1, ev2 = pca.explained_variance_ratio_
total_ev  = ev1 + ev2
print(f"Explained Variance  →  PC1: {ev1:.2%}  |  PC2: {ev2:.2%}  |  Total: {total_ev:.2%}")
print()
print("Kontribusi Fitur terhadap PC1 & PC2:")
loading_df = pd.DataFrame(
    pca.components_.T,
    index   = list(FEATURE_LABELS.values()),
    columns = ['PC1 Loading', 'PC2 Loading']
).round(3)
print(loading_df.to_string())
```

```
Explained Variance  →  PC1: 31.91%  |  PC2: 26.23%  |  Total: 58.13%

Kontribusi Fitur terhadap PC1 & PC2:
                             PC1 Loading  PC2 Loading
Jml Kejahatan\nDilaporkan          -0.510       -0.675
Risiko per\n100k Penduduk          -0.259        0.034
% Penyelesaian\nKasus              -0.454        0.737
Selang Waktu\nKejadian (jam)        0.683       -0.001
```

```
# ── Matplotlib: K-Means vs Hierarchical — Side-by-side ──────────────────────
col_list = sns.color_palette('husl', OPTIMAL_K)
titles   = ['K-Means Clustering', "Ward's Hierarchical Clustering"]
cluster_cols = ['Cluster_KMeans', 'Cluster_HC']

fig, axes = plt.subplots(1, 2, figsize=(20, 8))
for ax_i, (ax, col_key, title) in enumerate(zip(axes, cluster_cols, titles)):
    for cl_id in range(OPTIMAL_K):
        mask = df_clean[col_key] == cl_id
        ax.scatter(X_pc[mask, 0], X_pc[mask, 1],
                   s=160, alpha=0.88, color=col_list[cl_id],
                   edgecolors='white', linewidths=0.9,
                   label=f'Cluster {cl_id + 1}', zorder=4)
        # Anotasi nama daerah
        for idx in df_clean.index[mask]:
            row_num = df_clean.index.get_loc(idx)
            ax.annotate(
                df_clean.loc[idx, 'Kabupaten_Kota'],
                (X_pc[row_num, 0], X_pc[row_num, 1]),
                fontsize=7, ha='center', va='bottom',
                alpha=0.75, xytext=(0, 5), textcoords='offset points'
            )
    ax.set_title(f'{title}\n(PCA Projection 2D)', fontweight='bold')
    ax.set_xlabel(f'PC1  ({ev1:.1%} variance explained)')
    ax.set_ylabel(f'PC2  ({ev2:.1%} variance explained)')
    ax.legend(title='Cluster', framealpha=0.9)
    ax.grid(alpha=0.3, linestyle='--')

plt.suptitle(f'Visualisasi Cluster 2D via PCA — 38 Kabupaten/Kota Jawa Timur\n'
```

```
            f'Total Variance Explained: {total_ev:.1%}',
            fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('pca_clusters.png', bbox_inches='tight', dpi=130)
plt.show()
```




```
# ── Plotly Interactive: K-Means ──────────────────────────────────────────
df_plot = df_clean[['Kabupaten_Kota', 'Cluster_KMeans'] + FEATURE_COLS].copy()
df_plot['PC1']          = X_pc[:, 0]
df_plot['PC2']          = X_pc[:, 1]
df_plot['Cluster_Label'] = 'Cluster ' + (df_plot['Cluster_KMeans'] + 1).astype(str)

fig_px = px.scatter(
    df_plot,
    x='PC1', y='PC2',
    color='Cluster_Label',
    text='Kabupaten_Kota',
    hover_data={
        'Jml Kejahatan'    : df_plot['Jumlah_Kejahatan'].round(0),
        'Risiko/100k'      : df_plot['Risiko_100k'].round(1),
        '% Penyelesaian'   : df_plot['Persen_Penyelesaian'].round(2),
        'Selang Waktu (jam)': df_plot['Selang_Waktu'].round(0),
        'PC1'              : False,
        'PC2'              : False,
        'Cluster_Label'    : False,
    },
    color_discrete_sequence=px.colors.qualitative.Bold,
    title=f'<b>Interactive PCA Scatter — K-Means Clustering (K={OPTIMAL_K})</b><br>'
          f'<sup>Hover untuk detail | Total variance: {total_ev:.1%}</sup>',
    template='plotly_white',
    width=950, height=600,
)
fig_px.update_traces(
    textposition='top center',
    marker=dict(size=11, opacity=0.85, line=dict(width=1, color='white')),
)
fig_px.update_layout(
    font=dict(family='Open Sans', size=12),
    legend_title_text='Cluster',
    xaxis_title=f'PC1 ({ev1:.1%} variance)',
    yaxis_title=f'PC2 ({ev2:.1%} variance)',
)
fig_px.show()
```

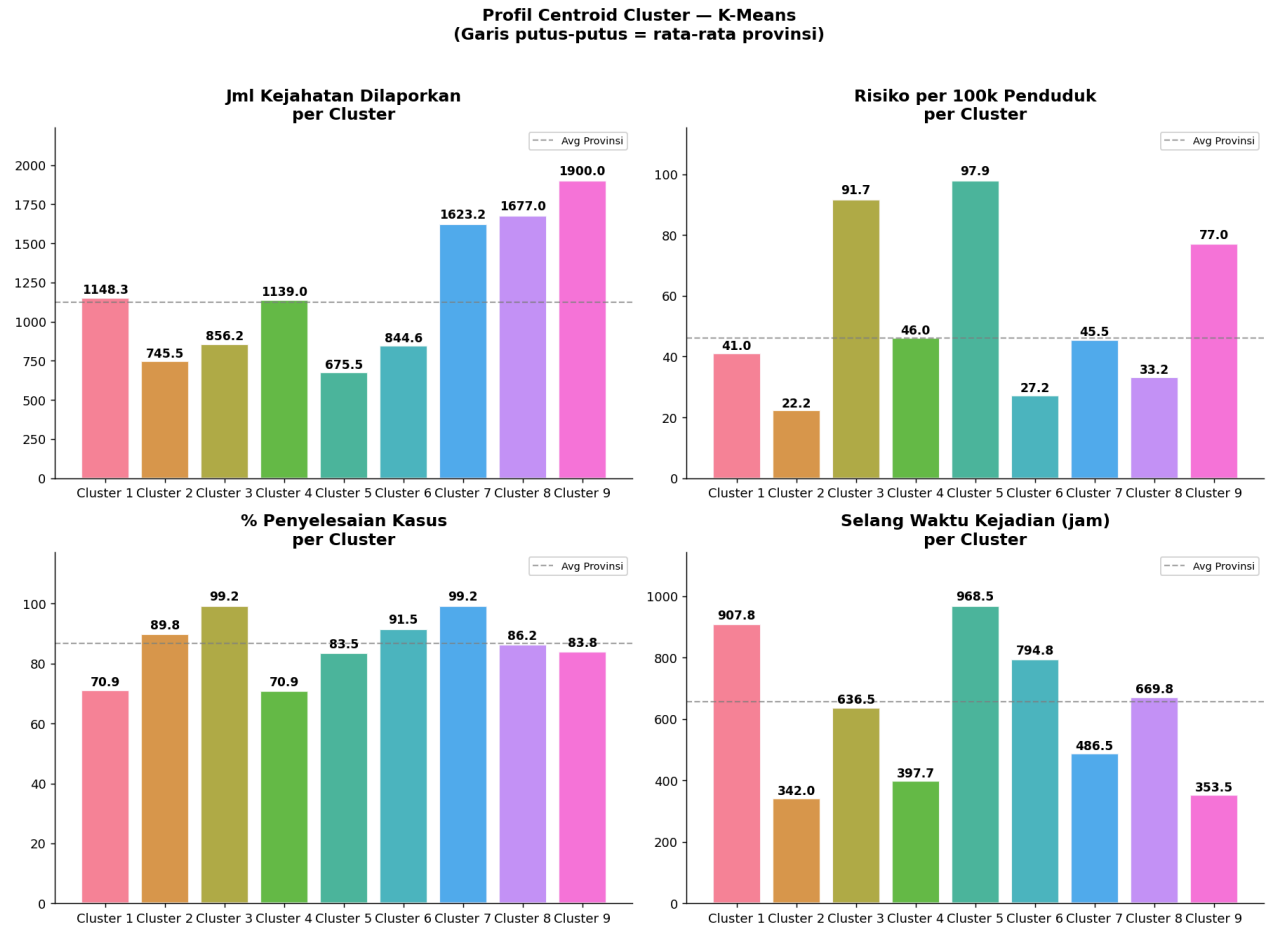
8. Profil Cluster & Pelabelan Risiko

8.1 Tabel Centroid & Statistik Cluster

```
# ── Tabel Profil (centroid = rata-rata per cluster) ──────────────────────
profile_tbl = (df_clean
               .groupby('Cluster_KMeans')[FEATURE_COLS]
               .agg(['mean', 'std', 'count']))

# Tampilan ringkas: rata-rata
centroid_tbl = df_clean.groupby('Cluster_KMeans')[FEATURE_COLS].mean().round(2)
centroid_tbl.index = [f'Cluster {i+1}' for i in centroid_tbl.index]
centroid_tbl.columns = ['Jml Kejahatan', 'Risiko/100k', '% Penyelesaian', 'Selang Waktu']
centroid_tbl['N Daerah'] = df_clean.groupby('Cluster_KMeans').size().values

print("=" * 72)
print("  TABEL CENTROID CLUSTER — K-MEANS")
print("=" * 72)
print(centroid_tbl.to_string())
```

```
========================================================================
  TABEL CENTROID CLUSTER — K-MEANS
========================================================================
            Jml Kejahatan  Risiko/100k  % Penyelesaian  Selang Waktu  N Daerah
Cluster 1        1148.33        41.00           70.92        907.83         6
Cluster 2         745.50        22.25           89.76        342.02         4
Cluster 3         856.25        91.66           99.20        636.52         4
Cluster 4        1139.00        46.00           70.86        397.67         3
Cluster 5         675.50        97.88           83.52        968.50         2
Cluster 6         844.56        27.22           91.49        794.80         9
Cluster 7        1623.25        45.50           99.23        486.50         4
Cluster 8        1677.00        33.25           86.25        669.75         4
Cluster 9        1900.00        77.00           83.85        353.50         2
```

```
# ── Visualisasi Bar Chart Profil Cluster ──────────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()
col_list_bar = sns.color_palette('husl', OPTIMAL_K)
bar_labels   = list(FEATURE_LABELS.values())

for fi, (col, lbl) in enumerate(zip(FEATURE_COLS, bar_labels)):
    vals  = df_clean.groupby('Cluster_KMeans')[col].mean()
    idx   = [f'Cluster {i+1}' for i in vals.index]
    bars  = axes[fi].bar(idx, vals.values,
                         color=[col_list_bar[j] for j in range(OPTIMAL_K)],
                         edgecolor='white', linewidth=1.2, alpha=0.88)
    # Annotate values
    for bar, val in zip(bars, vals.values):
        axes[fi].text(bar.get_x() + bar.get_width()/2,
                      bar.get_height() * 1.01, f'{val:.1f}',
                      ha='center', va='bottom', fontsize=9.5, fontweight='bold')
    # Grand mean reference line
    axes[fi].axhline(df_clean[col].mean(), color='grey', linestyle='--',
                     linewidth=1.2, alpha=0.7, label='Avg Provinsi')
    axes[fi].set_title(f'{lbl.replace(chr(10), " ")}\nper Cluster', fontweight='bold')
    axes[fi].legend(fontsize=8)
    axes[fi].set_ylim(0, vals.max() * 1.18)

fig.suptitle('Profil Centroid Cluster — K-Means\n'
             '(Garis putus-putus = rata-rata provinsi)',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('cluster_profiles.png', bbox_inches='tight', dpi=130)
plt.show()
```



8.2 Pelabelan Profil Risiko Intuitif

Setiap cluster diberi label deskriptif berdasarkan **Danger Score komposit** yang mempertimbangkan seluruh dimensi:

Dimensi	Arah Berbahaya	Bobot
Jumlah Kejahatan	Tinggi	25 %
Risiko per 100k	Tinggi	25 %
% Penyelesaian	Rendah	25 %
Selang Waktu	Pendek (rendah)	25 %

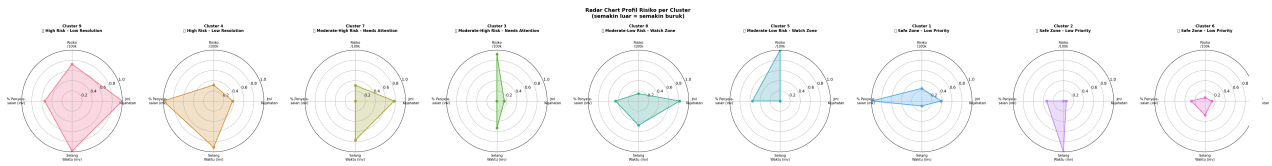
```
# ── Hitung Danger Score Komposit ──────────────────────────────────────
max_crime  = centroid_tbl['Jml Kejahatan'].max()
max_risk   = centroid_tbl['Risiko/100k'].max()
max_pct    = centroid_tbl['% Penyelesaian'].max()
max_selang = centroid_tbl['Selang Waktu'].max()

centroid_tbl['Danger_Score'] = (
    (centroid_tbl['Jml Kejahatan']   / max_crime)  * 25 +
    (centroid_tbl['Risiko/100k']     / max_risk)   * 25 +
    (1 - centroid_tbl['% Penyelesaian'] / max_pct) * 25 +
    (1 - centroid_tbl['Selang Waktu']  / max_selang) * 25
).round(2)

centroid_tbl_s = centroid_tbl.sort_values('Danger_Score', ascending=False).reset_index(drop=False)

# ── Assign Risk Labels berdasarkan rank Danger Score (dinamis untuk K apapun) ─
def get_risk_label(rank, total):
    pct = rank / max(total - 1, 1)
    if pct < 0.25:
```

```
        return '🔴 High Risk — Low Resolution'
    elif pct < 0.50:
        return '🟠 Moderate-High Risk — Needs Attention'
    elif pct < 0.75:
        return '🟡 Moderate-Low Risk — Watch Zone'
    else:
        return '🟢 Safe Zone — Low Priority'

n_clusters_actual = len(centroid_tbl_s)
centroid_tbl_s['Profil Risiko'] = [
    get_risk_label(i, n_clusters_actual) for i in range(n_clusters_actual)
]

print("=" * 85)
print("  TABEL PROFIL RISIKO CLUSTER — FINAL")
print("=" * 85)
display_cols = ['index', 'Jml Kejahatan', 'Risiko/100k',
                '% Penyelesaian', 'Selang Waktu', 'N Daerah',
                'Danger_Score', 'Profil Risiko']
print(centroid_tbl_s[display_cols].rename(columns={'index': 'Cluster'}).to_string(index=False))
```

```
=====================================================================================
  TABEL PROFIL RISIKO CLUSTER — FINAL
=====================================================================================
  Cluster  Jml Kejahatan  Risiko/100k  % Penyelesaian  Selang Waktu  N Daerah  Danger_Score
Cluster 9        1900.00        77.00           83.85        353.50         2         64.42          🔴 High Ris
Cluster 4        1139.00        46.00           70.86        397.67         3         48.62          🔴 High Ris
Cluster 7        1623.25        45.50           99.23        486.50         4         45.42 🟠 Moderate-High Risk
Cluster 3         856.25        91.66           99.20        636.52         4         43.25 🟠 Moderate-High Risk
Cluster 8        1677.00        33.25           86.25        669.75         4         41.54      🟡 Moderate-Low
Cluster 5         675.50        97.88           83.52        968.50         2         37.85      🟡 Moderate-Low
Cluster 1        1148.33        41.00           70.92        907.83         6         34.28          🟢 Safe Z
Cluster 2         745.50        22.25           89.76        342.02         4         34.05          🟢 Safe Z
Cluster 6         844.56        27.22           91.49        794.80         9         24.50          🟢 Safe Z
```

```
# ── Radar Chart / Spider Plot per Cluster ──────────────────────────────
from matplotlib.patches import FancyArrowPatch

# Normalisasi 0-1 untuk Radar
radar_data = centroid_tbl_s.set_index('index')[
    ['Jml Kejahatan', 'Risiko/100k', '% Penyelesaian', 'Selang Waktu']
].copy()

# Risiko dan kejahatan: min=baik, max=buruk → normalize as-is
# Penyelesaian dan Selang Waktu: max=baik → invert
radar_norm = radar_data.copy()
for col in radar_norm.columns:
    col_min, col_max = radar_norm[col].min(), radar_norm[col].max()
    radar_norm[col] = (radar_norm[col] - col_min) / (col_max - col_min + 1e-9)

# Invert "baik jika tinggi" metrics agar arah "luar" = lebih buruk
for good_col in ['% Penyelesaian', 'Selang Waktu']:
    radar_norm[good_col] = 1 - radar_norm[good_col]

categories  = ['Jml\nKejahatan', 'Risiko\n/100k', '% Penyele-\nsaian (inv)',
               'Selang\nWaktu (inv)']
N           = len(categories)
angles      = np.linspace(0, 2 * np.pi, N, endpoint=False).tolist()
angles     += angles[:1]

fig, axes_r = plt.subplots(1, n_clusters_actual,
                           figsize=(5 * n_clusters_actual, 5),
                           subplot_kw=dict(polar=True))
if n_clusters_actual == 1:
    axes_r = [axes_r]

for i, (cl_name, ax_r) in enumerate(zip(radar_norm.index, axes_r)):
    vals = radar_norm.loc[cl_name].tolist()
    vals += vals[:1]
    ax_r.plot(angles, vals, 'o-', linewidth=2, color=col_list[i], alpha=0.9)
    ax_r.fill(angles, vals, alpha=0.25, color=col_list[i])
    ax_r.set_xticks(angles[:-1])
    ax_r.set_xticklabels(categories, fontsize=8.5)
    ax_r.set_ylim(0, 1)
    risk_lbl = centroid_tbl_s.loc[
        centroid_tbl_s['index'] == cl_name, 'Profil Risiko'
    ].values[0]
    ax_r.set_title(f'{cl_name}\n{risk_lbl}', fontsize=9.5,
                   fontweight='bold', pad=15, y=1.12)

fig.suptitle('Radar Chart Profil Risiko per Cluster\n'
             '(semakin luar = semakin buruk)',
```

```
                fontsize=13, fontweight='bold', y=1.04)
plt.tight_layout()
plt.savefig('radar_chart.png', bbox_inches='tight', dpi=130)
```




```
# ── Peta Daerah per Cluster + Risk Label ──────────────────────────────────
# Gabungkan cluster label kembali ke df_clean untuk referensi
cluster_label_map = centroid_tbl_s.set_index('index')['Profil Risiko'].to_dict()
df_clean['Risk_Label'] = df_clean['Cluster_KMeans'].map(
    lambda x: cluster_label_map.get(f'Cluster {x+1}', 'Unknown')
)

# Print daftar daerah per profil risiko
print("\n📋  DAFTAR DAERAH BERDASARKAN PROFIL RISIKO")
print("=" * 70)
for cl_name, risk in cluster_label_map.items():
    cl_id  = int(cl_name.replace('Cluster ', '')) - 1
    members = df_clean.loc[df_clean['Cluster_KMeans'] == cl_id, 'Kabupaten_Kota'].tolist()
    print(f"\n  {risk}")
    print(f"  ({len(members)} Daerah): {', '.join(sorted(members))}")
```

```
📋  DAFTAR DAERAH BERDASARKAN PROFIL RISIKO
======================================================================

  🔴 High Risk — Low Resolution
  (2 Daerah): Gresik, Malang

  🔴 High Risk — Low Resolution
  (3 Daerah): Banyuwangi, Lumajang, Situbondo

  🟠 Moderate-High Risk — Needs Attention
  (4 Daerah): Jember, Kota Surabaya, Sidoarjo, Tulungagung

  🟠 Moderate-High Risk — Needs Attention
  (4 Daerah): Kota Mojokerto, Kota Probolinggo, Magetan, Ngawi

  🟡 Moderate-Low Risk — Watch Zone
  (4 Daerah): Jombang, Kediri, Lamongan, Pasuruan

  🟡 Moderate-Low Risk — Watch Zone
  (2 Daerah): Kota Kediri, Kota Pasuruan

  🟢 Safe Zone — Low Priority
  (6 Daerah): Bondowoso, Kota Malang, Mojokerto, Pamekasan, Sumenep, Tuban

  🟢 Safe Zone — Low Priority
  (4 Daerah): Blitar, Bojonegoro, Kota Blitar, Pacitan

  🟢 Safe Zone — Low Priority
  (9 Daerah): Bangkalan, Kota Batu, Kota Madiun, Madiun, Nganjuk, Ponorogo, Probolinggo, Sampang, Trenggalek
```